Two Boards, One Host: Running Muse Glimmer on a PCIe-Coupled Pair of Arduino Ventuno Q Computers

Christopher Maglio and Muse

October 31, 2026

Abstract

A single Arduino Ventuno Q carries 16 GB of LPDDR5 — just short of the 16.8 GB that Meta's official 4-bit build of Muse Glimmer occupies before any operating system, runtime, or KV cache is loaded. This paper describes a theoretical system in which two Ventuno Q boards are joined by a PCIe 4.0 x4 crossover cable between their M.2 slots, one board configured as the PCIe root complex (master) and the other as a PCIe endpoint presenting a Non-Transparent Bridge (client), so that the pair presents to inference software as one 32 GB host. We enumerate everything that must be created to build such a system — the cable power modification, the clock and reset role assignment, the endpoint-mode device tree and kernel work, the microcontroller coordination firmware, and the inference-layer integration — with a probable implementation route for each item. We then calculate the performance the finished system could be expected to deliver on the released builds of Glimmer, using a memory-bandwidth model calibrated against a measured dual-Tesla-P100 build published on video, whose performance techniques (speculative decoding above all) we analyze and map onto this system. The headline result: the pair is capacity-feasible at 4-bit and 6-bit, infeasible at 8-bit, and bandwidth-limited to roughly 2.4 tokens/second at 4-bit before speculative decoding, with a plausible 4–5.5 tokens/second once Glimmer's DFlash drafter is applied. All performance figures in this paper are calculations or labelled extrapolations, not measurements; the system described has not been built.

1. Introduction

The Arduino Ventuno Q is an unusual computer: a Qualcomm Dragonwing IQ-8275 (QCS8275) application processor with 16 GB of LPDDR5, paired on the same board with an STM32H5F5 real-time microcontroller, at a street price in the $299–399 range. It is nearly, but not quite, a Muse Glimmer machine. Glimmer is Meta's 30B-parameter open-weight model, released 2026-08-10 under Apache 2.0 and distilled from Muse Spark. Meta's official GGUF builds are 4-bit: the Q4_K_M file is 16.8 GB. Sixteen gigabytes of file does not fit into sixteen gigabytes of RAM alongside an

operating system. The next step up in single-board capacity costs several times the Ventuno's price.

The observation behind this paper is that the Ventuno Q has a genuine PCIe port — an M.2 Key M socket wired PCIe Gen 4 x4 directly from the SoC — and PCIe is the bus that the rest of the industry uses to make several processors behave as one machine. Two boards, one cable, and a master/client PCIe relationship would pool the pair's memory into a single 32 GB space for inference purposes, at a combined hardware cost of roughly $600–800. This paper works out what that system requires and what it would be worth.

Nothing here has been built or measured. The paper's status is a design study: the hardware facts are drawn from published specifications, the software path from the mainline Linux kernel, and every performance number is a calculation whose assumptions are stated where the number appears.

2. The system as proposed

Two identical Ventuno Q boards are mounted side by side — the interconnect is a short ribbon, so the boards live in one enclosure, centimeters apart. An ADT-Link M2N-series PCIe 4.0 x4 TX/RX-swap jumper cable joins the M.2 socket of each board. The swap is essential: the cable crosses each side's transmit pairs onto the other side's receive pairs, which is what allows two slot connectors to face each other.

PCIe topology admits exactly one root complex per hierarchy. One board therefore remains the host (the master): its PCIe controller operates in root-complex mode and enumerates the link. The other board's controller is reconfigured as a PCIe endpoint (the client), presenting a Non-Transparent Bridge (NTB) function. NTB is the standard mechanism for host-to-host PCIe communication: each side sees a bridge device, and the two systems exchange data through memory windows (one side's DMA writes land in the other side's RAM) and doorbell registers (interrupts that signal new data). On top of NTB, Linux provides `ntb_netdev`, a network interface carried over the bridge; inference software that can distribute a model across machines over a socket — llama.cpp's RPC backend is the relevant example — then runs unmodified, at PCIe latency and bandwidth instead of LAN latency and bandwidth.

The two STM32H5 microcontrollers, one on each board, form a supervisory pair: they sequence power and reset, hold the bring-up handshake, and run recovery if either Linux system restarts. The data path never touches them.

Component	Specification (as published)
SoC (each board)	

	Qualcomm Dragonwing IQ-8275: 8-core Kryo Gen 6 CPU (2 Prime @ 2.35 GHz, 2 Gold @ 2.10 GHz, 4 Silver @ 1.95 GHz), Adreno 623 GPU, Hexagon NPU rated 40 dense TOPS
Memory (each board)	16 GB LPDDR5, 4x16-bit channels (64-bit bus), with link ECC
Real-time MCU (each board)	STMicroelectronics STM32H5F5, Cortex-M33 @ 250 MHz, 4 MB flash, 1.5 MB RAM
PCIe port (each board)	M.2 Key M 2230 socket, PCIe Gen 4 x4 from the SoC, non-bootable
Interconnect	ADT-Link M2N PCIe 4.0 x4 TX/RX-swap jumper cable, ~7.9 GB/s per direction, sub-microsecond latency class
Aggregate system	32 GB RAM (2 x 16 GB), 16 CPU cores, 2 x 40 TOPS NPU (not used by the inference path in §5), one logical host for inference

Table 1: Hardware inventory. Board specifications per Arduino and Qualcomm product literature [1][2].

3. Why PCIe, and not the Ethernet port

The boards already carry 2.5 GbE, and llama.cpp's RPC layer split would run over it today with no new hardware. The honest case for the PCIe link rests on three differences. First, bandwidth: 2.5 GbE moves ~0.31 GB/s; PCIe Gen 4 x4 moves ~7.9 GB/s, about 25x. Second, latency: a PCIe transaction to an adjacent slot completes in roughly 500 ns; a commodity Ethernet round trip through two operating-system network stacks costs 50–200 µs — two orders of magnitude more, plus jitter. Third, kind: NTB memory windows are memory semantics, not messages; a device writes directly into the peer's RAM. For the layer-split inference mode calculated in §5, the per-token traffic is one activation crossing, and even Ethernet latency would be tolerable. What Ethernet forecloses is every finer-grained distribution — tensor-style splits synchronize dozens of times per token and die on network-stack latency — and it leaves throughput two orders of magnitude below the memory systems it connects. The PCIe link is the only interconnect on the board whose latency class matches the memory it joins. This paper's performance section, however, claims only the layer-split mode that existing software supports; finer splits over NTB are identified as future work in §4 (item W8).

4. What must be created

Ten work items stand between two retail boards and the system of §2. None requires new silicon, a new protocol, or a new driver architecture; the largest items are configuration and integration work against mainline Linux facilities. They are listed in the order a build would meet them. Items W1–W2 are electrical, W3–W6 are SoC

software, W7 is microcontroller firmware, W8 is the inference layer, and W9–W10 are the physical build and its validation.

W1 — Cable power isolation. Both M.2 sockets source 3.3 V from their own board's regulator. A straight-through cable ties the two rails together: the regulators contend, and power sequencing at boot becomes undefined. The cable must carry signals only. Probable approach: interrupt the 3.3 V run at the ADT-Link adapter board — the M2N board exposes 3V3/12V break/inject pads for exactly this class of modification — while leaving every ground pin continuous. The grounds are not optional: the single-ended sideband signals (PERST#, CLKREQ#) are ground-referenced logic, the differential receivers have a common-mode window relative to local ground, and two separately powered chassis must equalize potential through the ground pins rather than through a receiver front end.

W2 — Clock and reset role assignment. Two independent boards have two independent PCIe reference clocks, and a slot's reset signal has a direction: a host drives PERST# to a device. In the paired system exactly one board may source the reference clock (or both controllers must be configured for SRIS — separate reference clocks with independent spread spectrum), and PERST# must flow from master to client only; the client must not drive its slot reset as though a card were installed in it. Probable approach: carry the master's REFCLK across the cable (the swap cable routes the clock pair alongside the data pairs), disable spread-spectrum clocking on the link, and express both decisions in the device tree of item W3; on the client, wire PERST# as an input to the endpoint controller, per the endpoint driver's binding.

W3 — Client endpoint-mode device tree and boot configuration. The client's PCIe controller must come up in endpoint mode from reset. On Qualcomm platforms this is a device-tree property: the controller node takes the endpoint configuration (`qcom,pcie-ep` family) rather than the root-complex one, and the board's boot firmware must leave the controller uninitialized instead of probing it as a host for an NVMe drive. Arduino's stock image does neither. Probable approach: author a device tree variant for the client board in which the M.2-facing controller is declared as an endpoint, with the clock arrangement of W2 and PERST# described as an input; ship it as a replacement DTB in the client's boot image. This item is the project's gating risk: endpoint mode is a documented capability of the controller family and its Linux driver, but it has not, to our knowledge, been demonstrated on this board.

W4 — Client kernel build. The client's kernel must carry the endpoint stack: the Qualcomm PCIe endpoint driver (`CONFIG_PCIE_QCOM_EP`), the Linux PCI endpoint framework, and the NTB endpoint function (`CONFIG_PCI_EPF_NTB`), all of which exist in mainline. Probable approach: rebuild the Ubuntu kernel for the platform with those options enabled; at runtime, instantiate the NTB function through the endpoint framework's configfs interface, declaring the BAR layout and memory-window sizes

the master will map. No new driver code is anticipated; the work is configuration, the device tree of W3, and debugging the link training of W6.

W5 — Master kernel NTB enablement. The master stays a conventional root complex. Its kernel needs the host side of the same machinery: the NTB core, the NTB transport, and `ntb_netdev` so the bridge appears as a network interface. Probable approach: enable the NTB modules in the master's kernel build; on enumeration of the client's NTB function, the standard drivers bind without modification. This is the lightest software item in the list.

W6 — Bring-up sequencing. An endpoint must be fully configured before the host enumerates, or the host scans an empty bus and the device is never seen. The pair therefore needs a defined power-on order: client boots, instantiates its NTB function, and signals readiness; only then does the master start the link (release PERST#, train, enumerate, bind). Probable approach: a readiness handshake between the two microcontrollers (item W7) gates the master's link start; on the master, link start and any later re-scan are scripted operations invoked by the supervisory layer, not left to boot-time chance.

W7 — Microcontroller coordination firmware. Custom firmware for the two STM32H5F5 microcontrollers, so that the sequencing of W6 and all recovery from it are automatic rather than manual. Required behaviors: (a) the power/boot handshake — the client MCU asserts READY only when its SoC reports the endpoint configured; the master MCU holds off enumeration until READY; (b) reset coordination — if either SoC restarts, the MCU pair re-runs the handshake and triggers re-enumeration in the correct order; (c) a cross-board watchdog heartbeat, so a hung Linux cannot leave the link half-trained indefinitely. The channel between the MCUs is a wire pair or UART; the firmware is a small state machine, and the division of labor mirrors a supervision pattern the authors have used elsewhere: the MCU owns power, reset, and veto; the Linux processor owns the harness and the network stacks. The MCUs carry no inference data.

W8 — Inference-layer integration. llama.cpp must be run in its distributed RPC mode: an RPC server on the client exposing its memory and compute, the main process on the master, the model split by layers across the two, with the RPC transport riding `ntb_netdev` so the socket traffic takes the PCIe link rather than the Ethernet port. Glimmer's DFlash speculative decoding runs under `llama-server`; the drafter file (1.63 GB) is placed on the master, and the acceptance behavior of draft-then-verify across a layer split must be confirmed (this combination is not a configuration we have seen reported). Probable approach: pin a llama.cpp build known to carry the `muse-glimmer` architecture and the `draft-dflash` path; set the layer split unevenly (the master also runs the harness, the drafter, and any vision projector); measure acceptance rates before trusting the §5.4 estimates. A tensor-

parallel backend writing directly to NTB memory windows — bypassing sockets — is the known upgrade path and is explicitly out of scope for the first build.

W9 — Mechanical and thermal integration. The ribbon cable constrains the boards to one enclosure, centimeters apart, with the M.2 connectors' limited mating life respected (fit once, strain-relieve, do not re-plug for convenience). Combined draw is modest (tens of watts class), but sustained CPU and LPDDR5 load in a closed box still needs a designed airflow path. Probable approach: a simple two-board chassis with a shared low-speed fan, the cable dressed in a single gentle fold, and both boards' antennas (needed only for setup and management) routed clear of the enclosure walls.

W10 — Validation harness. A short test program before any inference is attempted: (a) link training verification — the link must come up at Gen 4 x4, not silently at a reduced rate or width; (b) NTB throughput and latency measurement with the kernel's `ntb_perf` tool, checked against the §5 assumptions; (c) an output-parity test — the paired system must reproduce, token for token at temperature zero, the output of the same model run on a single host — followed by the failure-injection tests implied by W7 (restart each side; the pair must recover or fail loudly, never silently). Probable approach: shell scripts and the in-tree test tools; no custom test software beyond orchestration.

5. Calculated performance

5.1 Method

Single-stream decoding of a dense model is, to first order, a memory-streaming problem: each generated token requires reading essentially all of the model's weights once. The decode rate is therefore bounded by weights ÷ memory bandwidth, and real systems land at some efficiency below that bound. In the layer split used here the two boards process their layer ranges sequentially for each token — board A's layers, then board B's — so the times add: the pair behaves as one machine whose weights sit behind a single board's memory pipe. (The interconnect contributes one activation crossing per token: Glimmer's hidden size is 6656, so the crossing is ~13 KB and one sub-microsecond latency event against a token period of hundreds of milliseconds — six orders of magnitude below the noise, which is precisely what the PCIe link of §3 buys.)

The bandwidth input requires an assumption, stated plainly: Qualcomm's product brief documents the IQ-8275's memory as LPDDR5 on a 4x16-bit (64-bit) bus [2] but does not state a speed grade. We assume LPDDR5-6400, the common grade for this silicon family, giving a peak of 51.2 GB/s per board. For the realized-efficiency factor we take 65–80%: mature llama.cpp CPU decode on mobile LPDDR systems typically sustains in that band, while the calibration case of §5.4 — a community fork on 2016

GPUs — realized ~46%, which is why the band does not start at 100%. If the board's memory is in fact clocked lower, every figure in §5.3 scales down in proportion; the capacity conclusions do not change.

5.2 The released Glimmer builds

Meta's official GGUF repository contains exactly four files [3]: two 4-bit text builds, the vision projector, and the DFlash drafter. Meta publishes no official 6-bit or 8-bit GGUF. Those exist as community builds (Unsloth's dynamic series and the static quant ladders), at the sizes shown. The BF16 checkpoint (59.6 GB) is published as safetensors only.

Build	Source	File size	Notes
Q4_K_M	Meta (official)	16.8 GB	Meta-rated ~1.0% avg. degradation vs full precision
Dynamic Q4_K_XL	Meta (official)	19.7 GB	Meta-rated ~0.2% avg. degradation; aimed at 32 GB cards
mmproj (vision)	Meta (official)	1.4 GB	Required only for image input
DFlash drafter	Meta (official)	1.63 GB	Speculative-decoding draft model (see §5.4)
Q6_K	Community	~23.0 GB	Static quant ladders; Unsloth dynamic equivalents similar
Q8_0	Community	~29.7 GB	Largest published GGUF quant
BF16	Meta (safetensors)	59.6 GB	Not a GGUF; vLLM route

Table 2: Released Glimmer builds and sizes [3][4][5].

5.3 Capacity and decode rate on the pair

Capacity is checked per board: an even layer split puts half the weights on each board, and each board must also carry its operating system, the inference runtime, and its share of the KV cache — a reserve of ~2 GB per board. Glimmer's KV cache is small by design (grouped-query attention with 2 KV heads and sliding-window attention on most layers); independent measurement on a DGX Spark put it at roughly 31 KB/token at FP16, i.e. ~1.0 GB at 32K context and ~4.1 GB at the full 131K, halved again with an 8-bit KV cache [6]. The DFlash drafter (1.63 GB) is charged to the master.

Build		Fits 2 x 16 GB?			

	Per-board weights		Decode ceiling	Expected decode	With DFlash (est.)
Q4_K_M (official)	8.4 GB	Yes, comfortably	3.05 tok/s	2.0–2.4 tok/s	3.4–5.3 tok/s
Q4_K_XL (official)	9.9 GB	Yes	2.60 tok/s	1.7–2.1 tok/s	2.9–4.6 tok/s
Q6_K (community)	11.5 GB	Yes	2.23 tok/s	1.5–1.8 tok/s	2.5–3.9 tok/s
Q8_0 (community)	14.9 GB	**No** — exceeds per-board budget once the ~2 GB reserve is charged	(1.73 tok/s)	—	—
BF16	29.8 GB	**No** — would need four boards	—	—	—

Table 3: Capacity and calculated decode rates. Ceiling = file size ÷ 51.2 GB/s (§5.1 assumption). Expected = ceiling at 65–80% realized efficiency. DFlash column applies the ×1.7–2.2 speculative factor derived in §5.4 and is an extrapolation, not a measurement.

The 8-bit result deserves emphasis because it is a capacity fact, not a speed estimate: the pair cannot hold Q8_0 at all. Anyone whose requirement is 8-bit Glimmer needs a different machine (a 64 GB-class single board runs it at an estimated 5–6 tok/s); the paired Ventunos are a 4-bit and 6-bit system.

5.4 Calibration: the dual-P100 build, and which of its tricks transfer

The closest published relative of this design is a video build [7]: two NVIDIA Tesla P100 PCIe cards (16 GB each, bought at $78 apiece) in a Dell Precision T5810 workstation, running Qwen3.8-27B — a dense 27B model of Glimmer's class — at Q6_K under llama.cpp. Its measured results: 29.7 tok/s with speculative decoding off; 49.2 tok/s median on ordinary writing and 66.7 tok/s on a reasoning puzzle with it on; ~400 tok/s prompt processing; 37 tok/s sustained at a full 128K context; the builder later reported 74 tok/s. As a check on §5.1's model: two P100s offer ~1464 GB/s of aggregate HBM2 bandwidth, so a ~22.5 GB Q6_K weight set has a tensor-split ceiling near 65 tok/s; the measured 29.7 with speculation off is ~46% of ceiling — the basis for the conservative efficiency band used in §5.3.

The build's performance comes from five techniques, of which three transfer:

1. **Speculative decoding (transfers; the largest single effect).** The Qwen GGUF carries a built-in multi-token-prediction (MTP) head: a small draft mechanism proposes several tokens, the full model verifies them in one pass, and accepted tokens amortize the weight-streaming cost that dominates decode. The measured gain was ×1.66 on writing and ×2.25 on reasoning text.

Glimmer's analogue is Meta's own DFlash drafter — a separate 1.63 GB file rather than a built-in head — for which Meta reports 74.9 → 233.4 tok/s (×3.1) on an RTX 5090 in llama.cpp. We deliberately do not apply the 5090 factor in Table 3: it was measured on a flagship GPU in a different regime. The ×1.7–2.2 band used there is the video's like-for-like software-speculation range, rounded conservatively.

2. **Tensor split across in-chassis PCIe (does not transfer — yet).** The P100s split every layer's matrices across both cards (`-sm tensor`) with peer-to-peer transfers enabled, so both cards stream weights concurrently. That mode exists inside one llama.cpp process driving local GPUs. Across two hosts, the available distribution is the RPC layer split of §5.1 — sequential, not concurrent — which is why the pair's ceiling is set by one board's bandwidth rather than two. A tensor-style backend over NTB memory windows is the single largest unrealized performance item in this design (see W8).
3. **Peer-to-peer hygiene (transfers in spirit).** The video's build runs with `iommu=pt` so peer traffic avoids IOMMU translation overhead. The NTB path has no IOMMU in it by construction — the window mapping is the direct path — but the same principle applies to keeping the RPC transport on `ntb_netdev` and off any virtualized or bridged interface.
4. **A pinned, platform-tuned inference build (transfers as practice).** The P100 results depend on a community fork of llama.cpp carrying Pascal-specific fixes; upstream alone did not produce them. The paired-Ventuno build should expect the same shape of dependency: a pinned llama.cpp version carrying the `muse-glimmer` architecture and a working `draft-dflash` path under `llama-server`, with upgrades treated as re-validation events (W10), not routine maintenance.
5. **Power and thermal discipline (transfers, scaled down).** The P100s are passive datacenter cards; the build caps them at 150 W each and ducts air across them, because unducted they reached 79 °C in 35 seconds. The Ventuno pair draws an order of magnitude less, but W9's designed airflow is the same lesson at smaller scale: sustained inference is a continuous load, and intermittent-load thermal design fails it.

5.5 Prefill: the weak direction

Prompt processing is compute-bound, and in a layer split it does not parallelize across the boards for a single sequence — the pair's prefill is one board's CPU at a time, in series. llama.cpp does not use the Hexagon NPU; the work falls to the Kryo cores and their integer dot-product NEON kernels. We estimate prompt processing in the 5–12 tok/s range (labelled estimate; the FLOPs-per-token of a 27.9B-parameter text model against the A78-class cluster's realistic sustained integer throughput). The consequence is workload-shaped: a 4,000-token document prompt costs on the order of ten minutes before the first answer token. For interactive chat

this is disqualifying; for the batch-shaped, precision-first document lookup the authors actually intend — where decode quality matters and prompts are ingested once, then queried — it is a real but bounded cost. Offloading prefill to the Adreno 623 through llama.cpp's Vulkan backend is the obvious investigation and is not counted anywhere in this paper.

5.6 Context

System	Best Glimmer build	Decode	Approx. cost
Single Ventuno Q	Official builds do not fit (≈14 GB usable < 16.8 GB)	—	$299–399
Paired Ventuno Q (this paper)	Q4_K_M / Q4_K_XL / Q6_K	2.0–2.4 tok/s calc.; 3.4–5.3 with DFlash (est.)	$600–800
Dual Tesla P100 workstation [7]	(Qwen3.8-27B Q6_K, measured)	29.7–66.7 tok/s	~$156 in cards + workstation
Vecow EAC-5100-R64 (AGX Orin 64 GB)	Q8_0	~5–6 tok/s (est.)	$1,999
DGX Spark, 128 GB (273 GB/s)	Q8_0 and above	Q4 ceiling ~16 tok/s class	$6,950

Table 4: The pair in context. The P100 row is a different model measured, not calculated, and is included as the method's calibration case, not as a like-for-like competitor.

6. Risks and open questions

The gating risk is W3/W4. Endpoint mode is a documented capability of the controller family — the silicon is a DesignWare-based dual-role design and mainline Linux carries a Qualcomm endpoint driver [8] — and a published precedent exists for the general pattern (an ARM single-board computer run as a PCIe endpoint, with an NTB function, over an ADT-Link swap cable [10]). But it has not been demonstrated on the Ventuno Q, and Arduino's kernel sources and device trees must be obtainable and rebuildable for W3–W5 to be practical at all. The project's first milestone is therefore deliberately small and binary: client configured as endpoint, cable attached, link trains, NTB function enumerates on the master. Everything else in §4 is downstream of that afternoon's result.

Link rate. The link may train below Gen 4 x4 — clock arrangement (W2) and ribbon signal integrity decide that. The design is unusually forgiving here: layer-split traffic needs kilobytes per token, so even a Gen 3 x1 fallback (~0.98 GB/s, ~1 µs class)

leaves every figure in §5.3 unchanged. Link rate would matter only to the future NTB-window backend of W8.

Assumption exposure. §5.3's decode figures scale linearly with the assumed LPDDR5-6400 speed grade; if the boards ship a slower grade, the rates fall in proportion. The capacity conclusions (Q4 and Q6 fit, Q8 does not) are independent of that assumption. The DFlash column is the softest number in the paper: speculative decoding over an RPC layer split, with the drafter on the far side of a split from some of the layers it drafts for, is a combination we have not seen reported, and acceptance behavior there is an empirical question (W8).

Software drift. Glimmer support in llama.cpp is young (the architecture landed the week of the model's release), and the P100 precedent (§5.4, item 4) suggests the working configuration will be a pinned build with known behavior rather than a rolling latest. The design treats the inference stack as a fixed component once validated.

7. Conclusion

Two $299–399 boards, one crossover cable, and ten enumerated work items produce a 32 GB Glimmer host for roughly $600–800 — the least expensive configuration we have identified that runs Meta's official Glimmer builds at all. Its character is set by two facts: capacity, which admits the 4-bit official builds and the community 6-bit build and refuses 8-bit outright; and bandwidth, which prices decode at a calculated 2.0–2.4 tok/s at Q4_K_M, rising to an extrapolated 3.4–5.3 tok/s with DFlash speculative decoding — the technique that the calibration build of §5.4 proves out at every scale it has been tried. Prefill is slow and does not parallelize; the system suits patient, precision-first, batch-shaped work, not conversation. The deciding experiment is small: one board must first enumerate as the other's endpoint. Until that afternoon's work is done, this paper is a calculation, and it has tried to be exact about which of its numbers are arithmetic, which are assumptions, and which are extrapolations.

References

[1] Arduino. "Arduino Ventuno Q (ABX00181)" product documentation and datasheet. arduino.cc.

[2] Qualcomm. "Dragonwing IQ8 Series Platform Product Brief," 87-83839-1. docs.qualcomm.com. (Memory documented as LPDDR5, 4x16 channels.)

[3] Meta Superintelligence Labs. "Muse-Glimmer-30B-GGUF." Hugging Face, meta-models/Muse-Glimmer-30B-GGUF. File sizes per repository LFS metadata: Q4_K_M 16,756,683,904 B; Dynamic Q4_K_XL 19,653,960,832 B; mmproj 1,400,328,928 B; DFlash 1,631,208,128 B.

[4] Community GGUF quant ladders for Muse-Glimmer-30B (static quants: Q6_K 23.0 GB, Q8_0 29.7 GB). Hugging Face.

[5] Unsloth. "Muse-Glimmer-30B-GGUF" (dynamic UD-series quants and memory guidance). Hugging Face.

[6] AI-Girls Lab. "Qwen3.8-27B and Muse-Glimmer-30B, measured on one DGX Spark." ai-girls.org, 2026. (KV-cache measurement: Glimmer ~31 KB/token at FP16, rounded source figure.)

[7] "Qwen 3.8 27B at 67 tok/s on Two $78 Nvidia Tesla P100s." YouTube, https://youtu.be/JvLUoJYUjDM. Configuration and results per the video's published description: 2x Tesla P100 PCIe 16 GB, Dell Precision T5810, Qwen3.8-27B Q6_K with MTP head, Kmic-68 P100 fork of llama.cpp, tensor split, P2P enabled with iommu=pt, 150 W per-card cap.

[8] Linux kernel. "PCI Endpoint Framework" documentation, kernel.org; drivers/pci/controller/dwc/pcie-qcom-ep.c (Qualcomm PCIe endpoint controller driver, CONFIG_PCIE_QCOM_EP); PCI EPF NTB function and NTB subsystem (ntb_netdev, ntb_perf).

[9] ADT-Link. "PCIe 4.0 x4 TX-RX Jumper Cable" (M2N / NS-series, Tx-to-Rx signal swap, "suitable for Master-Slave applications"). adtlink.com.

[10] R. (reds.ch). "An ARM single board computer as PCIe card" — Linux PCIe endpoint framework and NTB function over an ADT-Link R22SS swap cable. blog.reds.ch.

[11] llama.cpp project. muse-glimmer architecture support (build b10353 and later); RPC backend; DFlash speculative decoding (--spec-type draft-dflash); Meta's reported DFlash result on RTX 5090: 74.9 to 233.4 tok/s.